

Numele si prenumele (cu MAJUSCULE): _____ Grupa: _____

Test: _____ Tema: _____ Colocviu: _____ FINAL: _____

Test de laborator - Arhitectura Sistemelor de Calcul

ianuarie 2025
Seria 13

- Nota maxima pe care o puteti obtine este 10.
- Nota obtinuta trebuie sa fie minim 5 pentru a promova, fara nicio rotunjire superioara.
- Orice tentativa de fraudă este considerata o incalcare a Regulamentului de Etica!

1 Partea 0x00: x86 - maxim 6p

Presupunem ca aveti acces la un executabil `exec`, pe care il inspectati cu `objdump -d exec`. In momentul in care rulati aceasta comanda, va opriti asupra urmatorului cod. Analizati-l si raspundeti intrebarilor de mai jos. Pentru fiecare raspuns in parte, veti preciza si instructiunile care v-au ajutat in rezolvare.

```
000011ad <f>:
11b1: 55                push    %ebp
11b2: 89 e5            mov     %esp,%ebp
11b4: 83 ec 10        sub     $0x10,%esp
11c1: c7 45 fc 00 00 00 00 movl    $0x0,-0x4(%ebp)
11c8: c7 45 f8 00 00 00 00 movl    $0x0,-0x8(%ebp)
11cf: eb 4b          jmp     121c <f+0x6f>
11d1: 8b 45 f8        mov     -0x8(%ebp),%eax
11d4: 8d 14 85 00 00 00 00 lea     0x0(,%eax,4),%edx
11db: 8b 45 08        mov     0x8(%ebp),%eax
11de: 01 d0          add     %edx,%eax
11e0: 8b 00          mov     (%eax),%eax
11e2: 39 45 10        cmp     %eax,0x10(%ebp)
11e5: 7f 31          jg      1218 <f+0x6b>
11e7: 8b 45 f8        mov     -0x8(%ebp),%eax
11ea: 8d 14 85 00 00 00 00 lea     0x0(,%eax,4),%edx
11f1: 8b 45 08        mov     0x8(%ebp),%eax
11f4: 01 d0          add     %edx,%eax
11f6: 8b 00          mov     (%eax),%eax
11f8: 39 45 14        cmp     %eax,0x14(%ebp)
11fb: 7c 1b          jl      1218 <f+0x6b>
11fd: 8b 45 f8        mov     -0x8(%ebp),%eax
1200: 8d 14 85 00 00 00 00 lea     0x0(,%eax,4),%edx
1207: 8b 45 08        mov     0x8(%ebp),%eax
120a: 01 d0          add     %edx,%eax
120c: 8b 10          mov     (%eax),%edx
120e: 89 d0          mov     %edx,%eax
1210: c1 e0 02        shl     $0x2,%eax
1213: 01 d0          add     %edx,%eax
1215: 01 45 fc        add     %eax,-0x4(%ebp)
1218: 83 45 f8 02     addl    $0x2,-0x8(%ebp)
121c: 8b 45 f8        mov     -0x8(%ebp),%eax
121f: 3b 45 0c        cmp     0xc(%ebp),%eax
1222: 7c ad          jl      11d1 <f+0x24>
1224: 8b 45 fc        mov     -0x4(%ebp),%eax
1228: c3             ret

00001229 <g>:
122d: 55                push    %ebp
122e: 89 e5            mov     %esp,%ebp
1230: 53              push    %ebx
1231: 83 ec 14        sub     $0x14,%esp
123f: 6a 0a          push    $0xa
1241: 6a 00          push    $0x0
1243: ff 75 0c        pushl   0xc(%ebp)
1246: ff 75 08        pushl   0x8(%ebp)
1249: e8 5f ff ff ff call    11ad <f>
124e: 83 c4 10        add     $0x10,%esp
1251: 89 45 f8        mov     %eax,-0x8(%ebp)
1254: d9 ee          fldz
1256: d9 5d f4        fstps   -0xc(%ebp)
1259: c7 45 f0 00 00 00 00 movl    $0x0,-0x10(%ebp)
1260: eb 26          jmp     1288 <g+0x5f>
1262: 8b 45 f0        mov     -0x10(%ebp),%eax
1265: 8d 14 85 00 00 00 00 lea     0x0(,%eax,4),%edx
126c: 8b 45 08        mov     0x8(%ebp),%eax
126f: 01 d0          add     %edx,%eax
1271: 8b 00          mov     (%eax),%eax
1273: 89 45 e8        mov     %eax,-0x18(%ebp)
1276: db 45 e8        fildl   -0x18(%ebp)
1279: d8 4d 10        fmul    0x10(%ebp)
127c: d9 45 f4        flds    -0xc(%ebp)
127f: de c1          faddp   %st,%st(1)
1281: d9 5d f4        fstps   -0xc(%ebp)
1284: 83 45 f0 01     addl    $0x1,-0x10(%ebp)
1288: 8b 45 f0        mov     -0x10(%ebp),%eax
128b: 3b 45 0c        cmp     0xc(%ebp),%eax
128e: 7c d2          jl      1262 <g+0x39>
1290: d9 45 f4        flds    -0xc(%ebp)
1293: 8b 5d fc        mov     -0x4(%ebp),%ebx
1297: c3             ret
```

a. (0.75p) Cate argumente primeste procedura `f` si cum ati identificat acest numar de argumente?

- b. (0.75p) Ce tip de date returneaza procedura `f` si cum ati identificat acest tip?
- c. (0.75p) In timp ce analizati executabilul, observati in sectiunea `.data` valoarea `0x40200000`. In timp ce cititi codul, va dati seama ca aceasta valoare este, de fapt, o reprezentare pe formatul `single` a unei valori rationale. Despre ce valoare este vorba?
- d. (0.75p) Va atrage atentia codificarea hexa a instructiunilor, si doriti sa vedeti care este semantica acestora. Analizati instructiunea `j1`. Stiind ca `0x1b = 27`, respectiv `0xad = -83`, determinati regula reprezentarii acestei instructiuni. Numim aceasta instructiune ca fiind de salt scurt, de ce?
- e. (0.5p) Procedura `f` contine o structura repetitiva. Identificati toate elementele acestei structuri: initializarea contorului, conditia de a ramane in structura, respectiv pasul de continuare (operatia asupra contorului).
- f. (1p) Analizati acum procedura `g`. Primul lucru pe care il observati sunt instructiunile specifice pentru a lucra cu `floating point`. Identificati `fldz` care incarca 0 peste stiva FPU (in `%st(0)`), `fildl op` care incarca intregul `op` ca `float` pe stiva FPU (in `%st(0)`), `fmulb op` care efectueaza pe formatul `float` operatia `%st(0) := %st(0) * op`, `faddb op1, op2` care efectueaza pe formatul `float` `op2 := op2 + op1` (in cazul `faddb %st, %st(1)` efectueaza `%st(1) += %st(0)` si efectueaza `pop`). Avand aceste informatii, determinati care este structura repetitiva si ce se calculeaza in acea structura.

- g. (0.5p) Considerati rescrierea instructiunilor pe stiva FPU din procedura **g** in SIMD. Care este echivalentul lor? (pentru liniile dintre adresele de memorie 1276 - 1281).
- h. (1p) Observati ca la adresa 1249, din procedura **g** se face un *call* imbricat in procedura **f**. Reprezentati configuratia stivei de apel, in momentul in care se obtine adancimea maxima, considerand reprezentarea incepand cu argumentele primite de **g**.

2 Partea 0x01: RISC-V - maxim 3p

- a. (0.75p) De ce RISC-V are un numar mult mai mare de registrii decat x86? Oferiti macar 2 motivatii.
- b. (0.75p) Se considera un procesor RISC-V minimal (RV32I) care nu implementeaza niciun mecanism de securitate. Care este efectul codului de mai jos daca se ruleaza pe un astfel de procesor? Descrieti fiecare pas de la inceputul pana la finalul executiei (Mentionati la fiecare pas cum se modifica registrii).

```
.data
x: .long 0x00f28293

.text
.global main
main:
la a0, x
jr a0

li a7, 93
li a0, 0
ecall
```

- c. (0.75p) Ce valoare va fi depozitata in **a0** in urma executiei urmatoarelor instructiuni, stiind ca **pc** este intial 0? Prezantati efectul fiecarei instructiuni.

```
auipc a0, 0x12345
srli a0, a0, 4
auipc a1, 0x10000
add a0, a0, a1
```

- d. (0.75p) Vrem sa facem salt in functia **func** folosind instructiunea **jal a7, func**. Ce modificari ar trebui aduse functiei astfel incat revenirea din functie sa se realizeze cu succes?

3 Partea 0x02: Performanta si cache - maxim 1p

- a. (0.5p) Un procesor are un pipeline cu 5 stadii (Fetch, Decode, Execute, Memory, Write-back). Se considera urmatoarea secventa de instructiuni:

```
lw a0, 0(gp)
add a1, a0, a2
sw a1, 4(gp)
```

Identificati hazardurile de date si tipul lor. Presupunem ca folosim stall-uri pentru a le rezolva (instructiunea dependinta trebuie sa finalize stadiul de write-back pentru ca instructiunea curenta sa poata intra in stadiul de execute). In acest context, care este numarul total de cicluri in care se termina de executat intreaga secventa? (Pentru usurinta puteti realiza un tabel cu ciclurile.)

- b. (0.5p) Un sistem are o memorie principală de 2^{16} bytes iar cache-ul are o capacitate totală de 4 KB, cu o dimensiune a unui bloc de 64 bytes (atât pentru memoria principală, cât și pentru cache). Calculați numărul total de blocuri din memoria principală. Determinați numărul de linii (blocuri) din cache. În cazul unei scheme de mapare directă, unde va fi mapată adresa 0x2F3C în cache?

